Résoudre le Job Shop par essaims particulaires

Explication du code du projet `jsp_pso`

Groupe B — Optimisation combinatoire, HEH 2026–2027

Résumé

Ce document explique, fichier par fichier, ce que fait le programme `jsp_pso` : comment il lit une instance, comment une *particule* (une simple liste de nombres) devient un planning, comment l'essaim améliore ces plannings au fil du temps, et comment on l'empêche de stagner. Chaque mécanisme est illustré sur le petit atelier 3×3 des slides du cours, puis mesuré sur des instances de référence (JSPLIB). Le solveur est un PSO *pur* : aucune recherche locale n'y est ajoutée.

Table des matières

1 Le problème : qui passe en premier sur la machine ? 2
- 1.1 L'atelier . . . 2
- 1.2 Même atelier, deux ordres, deux résultats . . . 2
- 1.3 Pourquoi une métaheuristique ? . . . 2

2 Vue d'ensemble du programme 3

3 Lire l'instance (`instance.cpp`) 4
- 3.1 Dans le code . . . 4

4 Représenter une solution : les clés aléatoires 6

5 Décoder une particule (`schedule.cpp`) 7
- 5.1 Le décodeur semi-actif . . . 7
- 5.2 Le décodeur Giffler-Thompson paramétré (par défaut) . . . 7

6 L'essaim particulaire (`pso.cpp`) 11
- 6.1 Le principe . . . 11
- 6.2 Les réglages retenus . . . 12
- 6.3 La boucle principale . . . 14

7 Empêcher l'essaim de stagner 14
- 7.1 La topologie en anneau . . . 14
- 7.2 Le redémarrage partiel . . . 15
- 7.3 CLPSO : l'apprentissage global . . . 16
- 7.4 Le multi-essaims hétérogène . . . 18

8 Parallélisme et fiabilité 19

9 Résultats 22
- 9.1 Convergence sur l'instance des slides . . . 22
- 9.2 Ce qu'apporte chaque mécanisme . . . 23
- 9.3 Résultats par instance . . . 23
- 9.4 Ce que nous avons appris . . . 23

10 Utilisation **24**

1 Le problème : qui passe en premier sur la machine ?

1.1 L'atelier

Un *job shop* est un ensemble de **jobs** à exécuter sur un ensemble de **machines**. Chaque job suit une *recette* fixe : une suite d'opérations, chacune sur une machine précise, pendant une durée précise. Trois contraintes doivent être respectées :

1. **ordre imposé** : les étapes d'un même job se font dans l'ordre de la recette ;
2. **une machine, une opération à la fois** ;
3. **pas d'interruption** : une étape commencée va jusqu'au bout.

Les recettes ne sont pas négociables. **La seule décision, c'est l'ordre** dans lequel chaque machine traite les jobs qui l'utilisent. L'objectif est de minimiser le **makespan**, l'instant où le dernier job se termine.

TABLE 1 – L'exemple des slides : 3 jobs, 3 machines (machine · durée).

	Étape 1	Étape 2	Étape 3
Job 0	M0 · 3	M1 · 2	M2 · 4
Job 1	M1 · 4	M2 · 3	M0 · 2
Job 2	M2 · 2	M0 · 5	M1 · 3

1.2 Même atelier, deux ordres, deux résultats

Plaçons les opérations une par une dans un ordre donné. Chaque opération démarre *au plus tôt* : quand son job est prêt **et** que sa machine est libre.

```
début = max(job prêt, machine libre)    fin = début + durée
```

L'ordre A ci-dessous est la suite $J_1, J_0, J_2, J_2, J_0, J_1, J_1, J_2, J_0$: chaque lettre désigne « la prochaine étape de ce job ». Prenons la cinquième opération placée, J_0 étape 2 sur M1 :

```
J0 a fini son étape 1 à   3
M1 est occupée par J1 jusqu'à 4
début = max(3, 4) = 4   ⇒ J0 attend que la machine se libère
fin   = 4 + 2 = 6
```

1.3 Pourquoi une métaheuristique ?

Pour ce petit atelier, il existe $\frac{9!}{3!\,3!\,3!} = 1\,680$ ordres de placement. Nous les avons tous énumérés : le meilleur donne **11**, le pire **28**, et 384 ordres atteignent l'optimum. Pour 10 jobs et 10 machines (l'exemple `10_10` des slides), le nombre d'ordres monte à $\frac{100!}{(10!)^{10}} \approx 2{,}4 \times 10^{92}$. Il est impossible de tout essayer : il faut une méthode qui *cherche intelligemment*. C'est le rôle de l'optimisation par essaims particulaires.

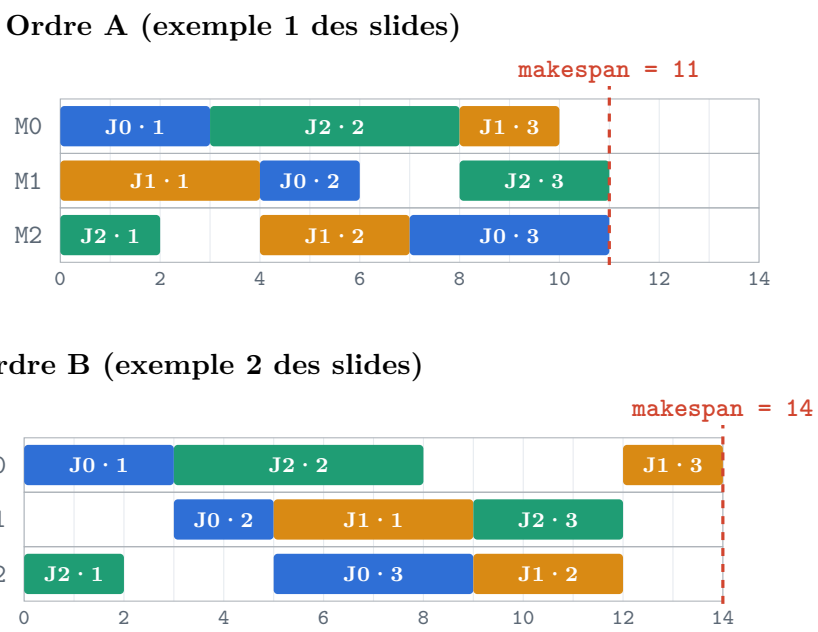


FIGURE 1 – Les mêmes recettes avec deux ordres de placement. Dans l'ordre B, J_0 passe en premier sur M1 : J_1 doit attendre jusqu'à 5 et tout le reste glisse (*effet domino*). Le makespan passe de 11 à 14.

2 Vue d'ensemble du programme

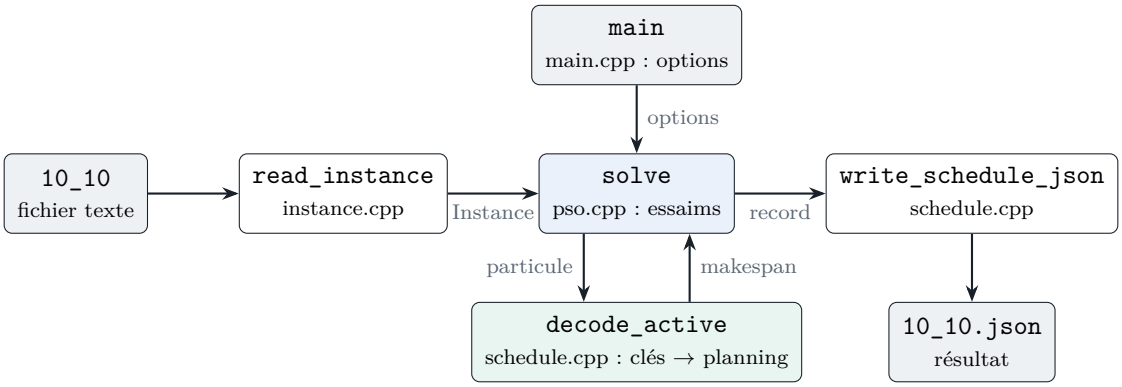


FIGURE 2 – Le trajet des données dans le programme.

TABLE 2 – Les fichiers du projet.

Fichier	Rôle
`src/instance.cpp`	lit le fichier d'instance et vérifie qu'il est bien formé
`src/schedule.cpp`	les deux décodeurs (particule → planning) et l'écriture du JSON
`src/pso.cpp`	l'essaim : mise à jour des particules, topologies, redémarrage, multi-essaims
`src/main.cpp`	lit les options de la ligne de commande et lance le solveur
`tests/test_core.cpp`	tests automatiques (lecture, exemple du cours, PSO)
`tools/validate.py`	vérificateur indépendant d'un JSON produit

3 Lire l'instance (`instance.cpp`)

Le fichier d'entrée a le format des slides : un nom, puis le nombre de jobs et de machines, puis une ligne par job contenant les couples `machine durée`.

```
10_10
10 10
4 88 8 68 6 94 5 99 1 67 2 89 9 77 7 99 0 86 3 92
...
```

En mémoire, l'atelier devient une structure `Instance` : le nom, `jobs`, `machines`, et `operations[job][étape]` qui contient le couple `(machine, duration)`.

La lecture est volontairement **robuste**, car une erreur de lecture le jour du concours coûterait une instance entière :

— les lignes vides, les commentaires `#` et les fins de ligne Windows (`\r`) sont acceptés ;

— si la première ligne contient déjà les dimensions, le nom est pris dans le nom du fichier (le JSON de sortie doit porter le même nom que l'instance) ;

— un fichier mal formé est refusé avec un message clair : couple manquant ou en trop, machine inexistante, machine utilisée deux fois par un même job, job manquant ;

— des lignes en trop après le dernier job déclenchent seulement un avertissement.

3.1 Dans le code

`src/instance.hpp`, lignes 6 à 16 — la structure qui représente l'atelier

```
6  struct Operation {
7      int machine = 0;
8      int duration = 0;
9  };
10
11 struct Instance {
12     std::string name;
13     int jobs = 0;
14     int machines = 0;
15     std::vector<std::vector<Operation>> operations;
16 };
```

l. 6–9 une `Operation` = une étape de recette : sur quelle machine, pendant combien de temps.

l. 15 `operations[job][étape]` : un tableau de recettes. Pour l'exemple 3×3, `operations[1][0]` vaut `{machine 1, durée 4}` (première étape du job 1).

`src/instance.cpp`, lignes 33 à 61 — nettoyer le fichier et trouver le nom

```
33     // On nettoie d'abord le fichier pour accepter les commentaires et
34     // les lignes vides, y compris avec des fins de ligne Windows.
35     std::vector<std::string> lines;
36     std::string line;
37     while (std::getline(input, line)) {
38         if (!is_comment_or_empty(line)) {
39             lines.push_back(line);
40         }
41     }
42     if (lines.empty()) {
43         throw std::runtime_error("Instance vide: " + path);
44     }
45
46     Instance result;
47     std::size_t index = 0;
48     {
49         // Certaines instances JSPLIB ont un nom en premiere ligne, d'autres
```

```
50          // commencent directement par les dimensions jobs x machines.
51          std::istringstream first(lines[0]);
52          int a = 0;
53          int b = 0;
54          std::string extra;
55          if ((first >> a >> b) && !(first >> extra)) {
56              result.name = std::filesystem::path(path).stem().string();
57          } else {
58              result.name = trim(lines[0]);
59              ++index;
60          }
61      }
```

l. 37–41 on lit tout le fichier ligne par ligne, et on ne garde que les lignes utiles (la fonction `is_comment_or_empty` écarte les lignes vides et celles qui commencent par `#`).

l. 51–55 on essaie de lire deux entiers sur la première ligne. Si elle ne contient *que* deux entiers, c'est déjà la ligne des dimensions : le fichier n'a pas de nom.

l. 56 dans ce cas, le nom est celui du fichier sans extension (`stem`) : `instances/abz5` donnera `abz5.json`.

l. 58–59 sinon, la première ligne est le nom (`trim` retire les espaces et le `\r` Windows), et on passe à la ligne suivante.

`src/instance.cpp`, lignes 75 à 105 — lire chaque recette et la vérifier

```
75      for (int job = 0; job < result.jobs; ++job) {
76          if (index >= lines.size()) {
77              throw std::runtime_error("Ligne manquante pour le job " +
78                                       std::to_string(job));
79          }
80          std::istringstream row(lines[index++]);
81          std::vector<bool> machine_used(
82              static_cast<std::size_t>(result.machines), false);
83          for (int operation = 0; operation < result.machines; ++operation) {
84              Operation value;
85              if (!(row >> value.machine >> value.duration) ||
86                  value.machine < 0 || value.machine >= result.machines ||
87                  value.duration < 0) {
88                  throw std::runtime_error("Operation invalide pour le job " +
89                                           std::to_string(job));
90              }
91              // Dans un job shop, chaque job passe une seule fois par machine.
92              if (machine_used[value.machine]) {
93                  throw std::runtime_error(
94                      "Machine " + std::to_string(value.machine) +
95                      " utilisee deux fois par le job " + std::to_string(job));
96              }
97              machine_used[value.machine] = true;
98              result.operations[job].push_back(value);
99          }
100         std::string leftover;
101         if (row >> leftover) {
102             throw std::runtime_error("Trop de nombres pour le job " +
103                                      std::to_string(job));
104         }
105     }
```

l. 75 une ligne par job.

l. 81–82 `machine_used` retient les machines déjà vues dans cette recette.

l. 85–90 on lit un couple `machine durée`. Si la lecture échoue (pas assez de nombres, ou pas un nombre) ou si la machine n'existe pas, on arrête avec un message clair.

l. 92–96 un job ne peut pas passer deux fois par la même machine.

l. 98 l'étape est ajoutée à la recette du job.

l. 100–104 s'il reste des nombres sur la ligne, elle est trop longue : erreur.

4 Représenter une solution : les clés aléatoires

Le PSO a été inventé pour des problèmes *continus* : une particule y est un point de $\mathbb{R}^D$ qui se déplace avec une vitesse. Or un ordonnancement est *discret*. Le pont entre les deux mondes est le codage par **clés aléatoires**.

Idée clé

Une particule est une liste de $D = n \times m$ nombres réels : **une clé par opération**. La clé de l'étape o du job j est rangée à l'indice $j \cdot m + o$. Un *décodeur* transforme ces nombres en planning, ce qui donne un makespan.

Sur l'atelier 3×3, une particule contient donc 9 nombres. Avec le décodeur le plus simple (« semi-actif », section 5.1), on trie les clés par ordre croissant ; chaque clé triée désigne un job, et on obtient une séquence de placement.

TABLE 3 – Une particule qui code l'ordre A. Trier les clés redonne la séquence $J_1, J_0, J_2, J_2, J_0, J_1, J_1, J_2, J_0$, donc le planning de makespan 11.

Opération	J0 · 1	J0 · 2	J0 · 3	J1 · 1	J1 · 2	J1 · 3	J2 · 1	J2 · 2	J2 · 3
Indice $j \cdot m + o$	0	1	2	3	4	5	6	7	8
Clé	0,12	0,40	0,88	0,05	0,52	0,61	0,21	0,33	0,77
Rang après tri	2	5	9	1	6	7	3	4	8

L'intérêt est double : *n'importe quelle* liste de nombres donne un planning valide (les contraintes sont garanties par le décodeur), et l'on peut appliquer telles quelles les formules continues du PSO.

src/pso.cpp, lignes 29 à 45 — des clés à la séquence de jobs (décodeur « semi »)

```
29 std::vector<int> decode_keys(const Instance& instance,
30                              const std::vector<double>& keys) {
31     // Decodeur "semi" : le tri des cles donne une sequence de jobs ;
32     // l'indice divise par m donne le job.
33     std::vector<int> indices(keys.size());
34     for (std::size_t index = 0; index < indices.size(); ++index) {
35         indices[index] = static_cast<int>(index);
36     }
37     std::stable_sort(indices.begin(), indices.end(),
38         [&keys](int left, int right) { return keys[left] < keys[right]; });
39     std::vector<int> sequence;
40     sequence.reserve(indices.size());
41     for (int index : indices) {
42         sequence.push_back(index / instance.machines);
43     }
44     return sequence;
45 }
```

l. 33–36 `indices` $= 0, 1, \ldots, D-1$: un numéro par opération.

l. 37–38 on trie ces numéros *selon la valeur de leur clé*. Sur la table 3, on obtient $3, 0, 6, 7, 1, 4, 5, 8, 2$ (la clé 0,05 de l'indice 3 est la plus petite, etc.). `stable_sort` garde l'ordre des indices en cas d'égalité, pour que le résultat soit toujours le même.

l. 41–43 chaque indice divisé par m (division entière) donne son job : $3/3 = 1$, $0/3 = 0$, $6/3 = 2 \ldots$ On retrouve la séquence $1, 0, 2, 2, 0, 1, 1, 2, 0$ de l'ordre A.

5 Décoder une particule (`schedule.cpp`)

5.1 Le décodeur semi-actif

C'est le décodeur utilisé dans les exemples des slides : on prend la séquence de jobs obtenue par le tri, et on place chaque opération *au plus tôt* derrière celles déjà placées. C'est exactement le calcul début = max(job prêt, machine libre) de la figure 1.

decode_semi_active

```
job_prêt[j] ← 0 ; machine_libre[k] ← 0 ; suivante[j] ← 0
pour chaque job j dans la séquence faire
    (k, durée) ← operations[j][suivante[j]] ; suivante[j] ← suivante[j] + 1
    début ← max(job_prêt[j], machine_libre[k]) ; fin ← début + durée
    job_prêt[j] ← fin ; machine_libre[k] ← fin
retourner le plus grand fin   ▷ le makespan
```

`src/schedule.cpp`, lignes 20 à 38 — le décodeur semi-actif

```
20      std::vector<int> next(instance.jobs, 0);
21      std::vector<int> job_ready(instance.jobs, 0);
22      std::vector<int> machine_ready(instance.machines, 0);
23      for (int job : sequence) {
24          if (job < 0 || job >= instance.jobs || next[job] >= instance.machines) {
25              throw std::runtime_error("Sequence invalide");
26          }
27          const int operation = next[job]++;
28          const auto& task = instance.operations[job][operation];
29          // Une operation attend a la fois la fin du job precedent et la
30          // liberation de sa machine.
31          const int start = std::max(job_ready[job], machine_ready[task.machine]);
32          const int end = start + task.duration;
33          result.jobs[job].push_back({task.machine, start, end});
34          job_ready[job] = end;
35          machine_ready[task.machine] = end;
36          result.makespan = std::max(result.makespan, end);
37      }
38      return result;
```

l. 20–22 trois compteurs : la prochaine étape de chaque job, la date où chaque job est prêt, la date où chaque machine se libère. Tout commence à 0.

l. 27–28 `next[job]++` donne l'étape à placer *puis* passe à la suivante ; `task` est cette étape (machine et durée).

l. 31 c'est exactement l'encadré `début = max(job prêt, machine libre)` de la section 1.

l. 34–35 le job et la machine sont tous deux occupés jusqu'à `end`.

l. 36 le makespan est la plus grande date de fin rencontrée.

Son défaut : il ne remplit jamais un « trou » laissé sur une machine. Beaucoup de particules différentes donnent de mauvais plannings, et l'essaim perd son temps.

5.2 Le décodeur Giffler-Thompson paramétré (par défaut)

Le décodeur par défaut (`decode_active`) suit l'algorithme de Giffler et Thompson, avec un paramètre δ (Gonçalves *et al.*). Il construit le planning *chronologiquement* et ne consulte la particule qu'au moment d'un **conflit**. À chaque étape :

1. pour chaque job, on calcule le début au plus tôt de sa prochaine opération et sa fin ;
2. φ = la plus petite fin possible. Sa machine M^* est celle où se joue le prochain conflit ;

3. σ = le plus petit début possible sur M^* ;
4. les **candidats** sont les opérations sur M^* qui peuvent commencer dans la fenêtre $[\sigma,\ \sigma + \delta(\varphi - \sigma)]$;
5. **la particule décide** : le candidat dont la clé est la plus petite passe.

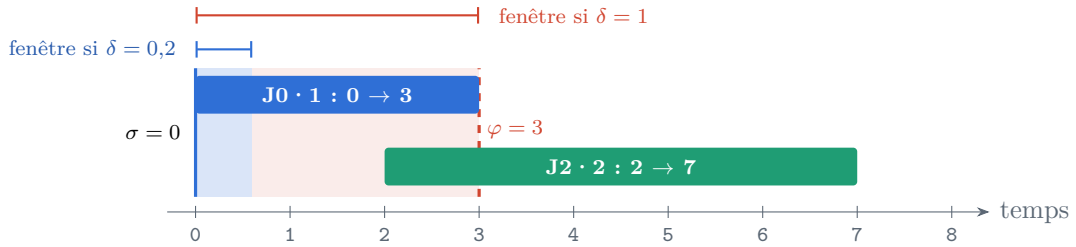


FIGURE 3 – Étape 2 du décodage de l'atelier 3×3 : J0 et J2 veulent tous deux la machine M0. La fenêtre réglée par δ fixe qui est candidat. Avec $\delta = 1$, les deux sont candidats et c'est la clé de la particule qui tranche ; avec $\delta = 0{,}2$, seul J0 peut commencer assez tôt (J2 ne commence qu'à $2 > 0{,}6$).

Le paramètre δ règle la taille de l'espace de recherche. Avec $\delta = 1$, on obtient des plannings *actifs* : on ne peut avancer aucune opération sans en retarder une autre, et l'optimum fait toujours partie de cet ensemble. Avec $\delta = 0$, on obtient des plannings *sans retard* : une machine n'attend jamais quand une opération est prête. Cet ensemble est plus petit et souvent très bon, mais il ne contient pas toujours l'optimum. La valeur retenue, $\delta = 0{,}2$, est un compromis mesuré expérimentalement (section 9).

TABLE 4 – Déroulé complet du décodeur ($\delta = 0{,}2$) sur l'atelier 3×3. Chaque case de candidat donne la machine et l'intervalle début–fin au plus tôt.

Pas	J0	J1	J2	φ (M^*)	σ	fenêtre	placé
1	M0 0–3	M1 0–4	M2 0–2	2 (M2)	0	[0; 0,4]	J2 : 0–2
2	M0 0–3	M1 0–4	M0 2–7	3 (M0)	0	[0; 0,6]	J0 : 0–3
3	M1 3–5	M1 0–4	M0 3–8	4 (M1)	0	[0; 0,8]	J1 : 0–4
4	M1 4–6	M2 4–7	M0 3–8	6 (M1)	4	[4; 4,4]	J0 : 4–6
5	M2 6–10	M2 4–7	M0 3–8	7 (M2)	4	[4; 4,6]	J1 : 4–7
6	M2 7–11	M0 7–9	M0 3–8	8 (M0)	3	[3; 4]	J2 : 3–8
7	M2 7–11	M0 8–10	M1 8–11	10 (M0)	8	[8; 8,4]	J1 : 8–10
8	M2 7–11	–	M1 8–11	11 (M2)	7	[7; 7,8]	J0 : 7–11
9	–	–	M1 8–11	11 (M1)	8	[8; 8,6]	J2 : 8–11

Sur ce petit exemple, toutes les fenêtres ne contiennent qu'un seul candidat : le décodeur seul construit directement le planning optimal (makespan 11), le même que l'ordre A de la figure 1. Sur une vraie instance 10×10, les conflits à plusieurs candidats sont nombreux, et ce sont eux que l'essaim apprend à trancher.

Dans le code

La boucle principale de `decode_active` fait une opération par tour (l. 70 : $n \times m$ tours en tout). Chaque tour se lit en trois temps, qui correspondent aux colonnes de la table 4.

src/schedule.cpp, lignes 74 à 89 — temps 1 : trouver φ, M^* et les σ

```
74          std::fill(machine_earliest.begin(), machine_earliest.end(),
75                    std::numeric_limits<int>::max());
76          int phi = std::numeric_limits<int>::max();
77          int conflict_machine = -1;
78          for (int job = 0; job < jobs; ++job) {
79              const int machine = current_machine[job];
80              if (machine < 0) continue;
81              const int begin = std::max(job_ready[job], machine_ready[machine]);
82              earliest[job] = begin;
83              machine_earliest[machine] = std::min(machine_earliest[machine], begin);
84              const int end = begin + instance.operations[job][next[job]].duration;
85              if (end < phi) {
86                  phi = end;
87                  conflict_machine = machine;
88              }
89          }
```

l. 74–75 on remet à « l'infini » le plus petit début connu de chaque machine.

l. 78–80 on parcourt les jobs ; `current_machine[job]` vaut -1 quand le job est terminé, on le saute.

l. 81 début au plus tôt de la prochaine étape du job : toujours le même `max(job prêt, machine libre)`.

l. 83 on retient au passage le plus petit début sur chaque machine : ce sera σ pour la machine du conflit.

l. 84–88 on cherche la plus petite date de fin : c'est φ, et sa machine devient `conflict_machine` (M^*). Au pas 1 du 3×3, c'est J2 sur M2 qui finit le plus tôt (2), donc $\varphi = 2$ et $M^* = \text{M2}$.

src/schedule.cpp, lignes 90 à 106 — temps 2 : la fenêtre et le choix de la particule

```
90          // delta regle la fenetre [sigma, sigma + delta (phi - sigma)] des
91          // candidats ; parmi eux, la particule decide : cle la plus petite.
92          const int sigma = machine_earliest[conflict_machine];
93          const double limit = sigma + delta * (phi - sigma);
94          int chosen = -1;
95          double chosen_priority = 0.0;
96          for (int job = 0; job < jobs; ++job) {
97              if (current_machine[job] != conflict_machine ||
98                  earliest[job] > limit) {
99                  continue;
100             }
101             const double priority = priorities[job * machines + next[job]];
102             if (chosen < 0 || priority < chosen_priority) {
103                 chosen = job;
104                 chosen_priority = priority;
105             }
106         }
```

l. 92–93 la fenêtre $[\sigma,\ \sigma + \delta(\varphi - \sigma)]$: `limit` en est la borne droite. Au pas 2 : $\sigma = 0$, $\varphi = 3$, $\delta = 0{,}2$, donc `limit` $= 0{,}6$.

l. 97–100 on écarte les jobs qui ne veulent pas M^*, ou qui ne peuvent pas commencer dans la fenêtre (au pas 2, J2 commence à $2 > 0{,}6$: il est écarté).

l. 101 la clé de la particule pour *cette* opération : indice `job * machines + next[job]`, comme dans la table 3.

l. 102–105 parmi les candidats, la plus petite clé gagne. **C'est la seule ligne où la particule influence le planning.**

`src/schedule.cpp`, lignes 107 à 115 — temps 3 : placer l'opération choisie

```
107         const int operation = next[chosen]++;
108         const int end = earliest[chosen] +
109                         instance.operations[chosen][operation].duration;
110         if (starts) (*starts)[chosen * machines + operation] = earliest[chosen];
111         job_ready[chosen] = end;
112         machine_ready[conflict_machine] = end;
113         current_machine[chosen] = next[chosen] < machines
114             ? instance.operations[chosen][next[chosen]].machine : -1;
115         makespan = std::max(makespan, end);
```

l. 107–109 l'opération choisie commence à son début au plus tôt ; on calcule sa fin.

l. 110 si l'appelant veut le planning complet (nouveau record), on note la date de début.

l. 111–112 le job et la machine sont occupés jusqu'à cette fin.

l. 113–114 le job passe à sa machine suivante, ou à -1 s'il a fini sa recette.

Vitesse. Le décodeur est appelé des millions de fois. Ses tableaux de travail sont réutilisés d'un appel à l'autre (`thread_local`) et il ne fait que deux passes sur les jobs par opération placée. Il ne construit le planning complet (`schedule_from_starts`) que lorsqu'un nouveau record est trouvé. Résultat : environ 170 000 évaluations par seconde sur un 10×10, et 46 μs par évaluation sur un 30×20.

`src/schedule.cpp`, lignes 53 à 59 — des tableaux de travail réutilisés

```
53      thread_local std::vector<int> next;
54      thread_local std::vector<int> job_ready;
55      thread_local std::vector<int> machine_ready;
56      thread_local std::vector<int> earliest;
57      thread_local std::vector<int> machine_earliest;
58      // current_machine[job] vaut -1 quand le job est termine.
59      thread_local std::vector<int> current_machine;
```

l. 53–59 `thread_local` : chaque thread garde ses propres tableaux d'un appel à l'autre. On évite ainsi de réserver de la mémoire des millions de fois, sans que deux essaims parallèles écrivent dans les mêmes tableaux.

`src/pso.cpp`, lignes 49 à 63 — évaluer une particule

```
49  int evaluate(const Instance& instance, const std::vector<double>& keys,
50               const SolverOptions& options, Schedule* schedule = nullptr) {
51      if (options.decoder == "semi") {
52          Schedule decoded = decode_semi_active(instance,
53                                                decode_keys(instance, keys));
54          const int makespan = decoded.makespan;
55          if (schedule) *schedule = std::move(decoded);
56          return makespan;
57      }
58      if (!schedule) return decode_active(instance, keys, options.delta);
59      std::vector<int> starts;
60      const int makespan = decode_active(instance, keys, options.delta, &starts);
61      *schedule = schedule_from_starts(instance, starts);
62      return makespan;
63  }
```

l. 51–57 avec `--decoder semi`, on trie les clés puis on applique le décodeur semi-actif.

l. 58 cas le plus fréquent : on ne veut que le makespan, sans construire le planning.

l. 59–61 pour un nouveau record seulement, on récupère les dates de début et on construit le planning complet qui ira dans le JSON.

6 L'essaim particulaire (`pso.cpp`)

6.1 Le principe

L'optimisation par essaims particulaires (Kennedy et Eberhart, 1995) imite une nuée d'oiseaux. Chaque particule i possède :

— une **position** x_i (ses D clés, donc un planning) ;

— une **vitesse** v_i ;

— son **meilleur souvenir** p_i (*pbest*), la meilleure position qu'elle a visitée ;

— un **guide social** g_i : le meilleur souvenir de ses voisins (*gbest* quand le voisinage est tout l'essaim).

À chaque tour, chaque dimension d de chaque particule est mise à jour :

$$v_{i,d} \leftarrow \underbrace{w\, v_{i,d}}_{\text{inertie}} + \underbrace{c_1 r_1 \,(p_{i,d} - x_{i,d})}_{\text{mémoire}} + \underbrace{c_2 r_2 \,(g_{i,d} - x_{i,d})}_{\text{influence sociale}}, \qquad x_{i,d} \leftarrow x_{i,d} + v_{i,d}$$

où r_1, r_2 sont tirés au hasard dans $[0, 1]$ à chaque fois.

`src/pso.cpp`, lignes 18 à 27 — une particule en mémoire

```
18 struct Particle {
19     std::vector<double> position;
20     std::vector<double> velocity;
21     std::vector<double> best_position;
22     int best_makespan = std::numeric_limits<int>::max();
23     // CLPSO : exemplar[d] est la particule dont le meilleur personnel guide
24     // la dimension d ; stale compte les iterations sans progres.
25     std::vector<int> exemplar;
26     int stale = 0;
27 };
```

l. 19–21 x_i, v_i et p_i : trois listes de D nombres.

l. 22 le makespan de p_i, pour ne pas le recalculer.

l. 25–26 utilisés seulement par la variante CLPSO (section 7.3).

`src/pso.cpp`, lignes 310 à 324 — la formule du PSO, telle qu'elle est codée

```
310                     const std::vector<double>& social =
311                         swarm[guide[index]].best_position;
312                     // Mise a jour PSO : inertie, attraction vers le meilleur
313                     // personnel puis vers le meilleur du voisinage.
314                     for (int key = 0; key < dimension; ++key) {
315                         double speed = inertia * particle.velocity[key] +
316                             c * unit(random) *
317                                 (particle.best_position[key] -
318                                  particle.position[key]) +
319                             c * unit(random) *
320                                 (social[key] - particle.position[key]);
321                         speed = std::clamp(speed, -max_velocity, max_velocity);
322                         particle.velocity[key] = speed;
323                         particle.position[key] += speed;
324                     }
```

l. 310–311 `social` est g_i, le meilleur souvenir du guide de la particule (choisi par la topologie, section 7.1).

l. 314 la mise à jour se fait dimension par dimension (`key` $= d$).

l. 315 $w\, v_{i,d}$: l'inertie.

l. 316–318 $c_1 r_1 (p_{i,d} - x_{i,d})$: la mémoire. `unit(random)` tire un nouveau r_1 dans $[0, 1]$ pour chaque dimension.

l. 319–320 $c_2 r_2 (g_{i,d} - x_{i,d})$: l'influence sociale.

l. 321 la vitesse est bornée à $\pm$`vmax`.

l. 322–323 on mémorise la vitesse et on déplace la particule.

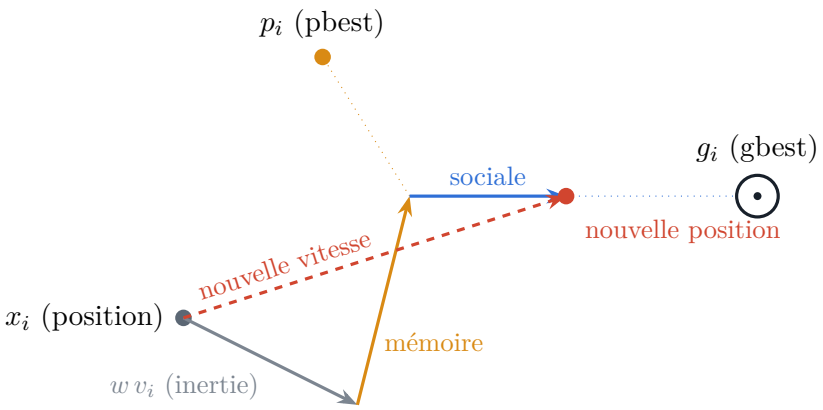


FIGURE 4 – La nouvelle vitesse est la somme de trois tendances : continuer sur sa lancée, revenir vers son meilleur souvenir, et aller vers le meilleur de ses voisins.

6.2 Les réglages retenus

— **Coefficients** $c_1 = c_2 = 1{,}49618$: les valeurs de « constriction » de Clerc et Kennedy, qui garantissent un essaim stable.

— **Inertie décroissante** de $w = 0{,}9$ à $w = 0{,}4$ au fil du temps : l'essaim explore au début et affine à la fin (`--no-decreasing-inertia` fixe $w = 0{,}7298$).

— **Vitesse bornée** à $\pm 0{,}2$ par dimension (`--vmax`) : une particule ne saute pas d'un bout à l'autre de l'espace en un tour.

— **Positions non bornées** : seul l'ordre relatif des clés compte. Si on bloquait les clés dans $[0, 1]$, beaucoup s'empileraient à 0 ou à 1, deviendraient égales, et la particule perdrait de l'information.

— **Plateaux acceptés** : une position de *même* makespan remplace le meilleur souvenir. Le JSP a de larges plateaux de solutions équivalentes ; sans cela, l'essaim s'y figerait.

— **Départ structuré** : dans le premier essaim, trois particules partent de règles de priorité classiques (SPT : opération la plus courte ; MWKR : job avec le plus de travail restant ; MOPNR : job avec le plus d'opérations restantes). Les autres partent au hasard.

`src/pso.cpp`, lignes 285 à 290 — l'inertie décroissante

```
285             const double elapsed =
286                 std::chrono::duration<double>(Clock::now() - start).count();
287             const double ratio = options.seconds == 0.0
288                 ? 0.0 : std::min(1.0, elapsed / options.seconds);
289             const double inertia = options.decreasing_inertia
290                 ? 0.9 - 0.5 * ratio : 0.7298;
```

l. 287–288 `ratio` va de 0 (début) à 1 (fin du budget).

l. 289–290 $w = 0{,}9 - 0{,}5 \times$`ratio` : 0,9 au départ, 0,4 à la fin. Avec `--no-decreasing-inertia`, w reste à 0,7298.

`src/pso.cpp`, lignes 179 à 189 — tirer une particule au hasard

```
179         const auto randomize = [&](Particle& particle) {
180             for (int index = 0; index < dimension; ++index) {
181                 particle.position[index] = unit(random);
182                 particle.velocity[index] = velocity(random);
183             }
184             particle.best_position = particle.position;
185             particle.best_makespan =
186                 evaluate(instance, particle.position, options);
187             particle.exemplar.clear();
188             particle.stale = 0;
189         };
```

l. 181 chaque clé est tirée uniformément dans $[0, 1]$.

l. 182 chaque vitesse est tirée dans $[-\texttt{vmax}, \texttt{vmax}]$.

l. 184–186 au départ, le meilleur souvenir est la position elle-même.

l. 187–188 remise à zéro de l'état CLPSO (utile après un redémarrage).

`src/pso.cpp`, lignes 72 à 89 — les règles de priorité du départ

```
72      const auto score = [&](int job) {
73          if (rule == 0) return -instance.operations[job][next[job]].duration;
74          int value = 0;
75          for (int operation = next[job]; operation < instance.machines;
76               ++operation) {
77              value += rule == 2 ? 1 : instance.operations[job][operation].duration;
78          }
79          return value;
80      };
81      for (int step = 0; step < instance.jobs * instance.machines; ++step) {
82          int selected = -1;
83          for (int job = 0; job < instance.jobs; ++job) {
84              if (next[job] >= instance.machines) continue;
85              if (selected < 0 || score(job) > score(selected)) selected = job;
86          }
87          sequence.push_back(selected);
88          ++next[selected];
89      }
```

l. 73 SPT : on préfère l'opération la plus courte (score $= -$durée, le plus grand score gagne).

l. 74–79 MWKR additionne les durées restantes du job ; MOPNR compte les étapes restantes.

l. 81–89 on construit une séquence en prenant à chaque pas le job au meilleur score. Les lignes 90 à 97 transforment cette séquence en clés (le rang divisé par D).

`src/pso.cpp`, lignes 226 à 240 — créer l'essaim

```
226         for (int index = 0; index < size; ++index) {
227             Particle& particle = swarm[index];
228             particle.position.resize(dimension);
229             particle.velocity.resize(dimension);
230             randomize(particle);
231             // Seul le premier essaim recoit les regles de priorite : les
232             // autres restent aleatoires pour diversifier les threads.
233             if (worker_id == 0 && index < 3) {
234                 particle.position = keys_from_priority(instance, index);
235                 particle.best_position = particle.position;
236                 particle.best_makespan =
237                     evaluate(instance, particle.position, options);
238             }
239             publish(particle.best_position, particle.best_makespan, worker_id);
240         }
```

l. 230 toutes les particules sont d'abord tirées au hasard.

l. 233–238 dans le premier essaim seulement, les particules 0, 1 et 2 sont remplacées par SPT, MWKR et MOPNR. Les autres essaims restent entièrement aléatoires, pour être plus divers.

l. 239 chaque particule propose son planning comme record global.

6.3 La boucle principale

Un essaim (un thread)

```
1  initialiser 30 particules (3 règles de priorité + hasard) ; évaluer chacune
2  tant que le temps n'est pas écoulé faire
3      w ← 0,9 - 0,5 × (temps écoulé / budget)
4      calculer le guide de chaque particule (voisinage en anneau ou CLPSO)
5      pour chaque particule i faire
6          mettre à jour v_i et x_i, dimension par dimension
7          f ← makespan(décoder(x_i))
8          si f ≤ f(p_i) alors p_i ← x_i ; publier si c'est un record global
9      si pas de progrès depuis 200 tours alors relancer l'essaim (sauf la meilleure)
10     tous les 50 tours : la pire particule adopte le record global (migration)
```

Les lignes 282 à 376 de `pso.cpp` codent cette boucle. La mise à jour des vitesses (lignes 7–8 du pseudo-code) a été montrée plus haut ; voici l'évaluation et la mémoire (lignes 9–10).

`src/pso.cpp`, lignes 326 à 339 — évaluer la nouvelle position et mettre à jour pbest

```
326                 const int makespan =
327                     evaluate(instance, particle.position, options);
328                 if (makespan < particle.best_makespan) {
329                     particle.stale = 0;
330                 } else {
331                     ++particle.stale;
332                 }
333                 // A makespan egal, on accepte quand meme le deplacement :
334                 // le JSP a de larges plateaux et cela evite de s'y figer.
335                 if (makespan <= particle.best_makespan) {
336                     particle.best_makespan = makespan;
337                     particle.best_position = particle.position;
338                     publish(particle.position, makespan, worker_id);
339                 }
```

l. 326–327 la nouvelle position est décodée en planning ; on récupère son makespan.

l. 328–332 `stale` compte les tours sans amélioration stricte (pour CLPSO).

l. 335 `<=` et non `<` : à makespan égal, la particule accepte quand même sa nouvelle position comme meilleur souvenir. Elle peut ainsi avancer sur un plateau.

l. 338 si c'est mieux que le record de tous les essaims, `publish` l'enregistre (section 8).

7 Empêcher l'essaim de stagner

Un PSO classique converge **trop vite** : toutes les particules s'agglutinent autour du premier bon planning trouvé, leurs vitesses tombent à zéro et plus rien ne bouge. Sur abz5, la version de départ ne progressait plus après une seconde, sur les 55 disponibles. Quatre mécanismes combattent ce phénomène. Ils restent tous des mécanismes d'essaim.

7.1 La topologie en anneau

Les particules sont disposées en cercle. Le guide social de chacune est le meilleur souvenir parmi elle-même et ses $k = 2$ voisins de chaque côté (`--neighbors`). Un bon planning se propage donc

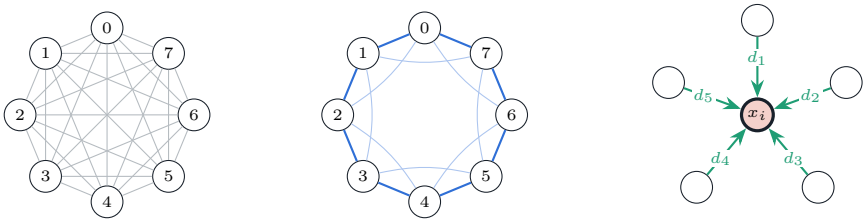


FIGURE 5 – Les trois façons de choisir le guide social d'une particule.

de proche en proche au lieu de contaminer tout l'essaim d'un coup : plusieurs zones de l'espace restent explorées en parallèle plus longtemps.

src/pso.cpp, lignes 245 à 271 — calculer le guide de chaque particule

```
245         const auto compute_guides = [&]() {
246             int best = 0;
247             for (int index = 1; index < size; ++index) {
248                 if (swarm[index].best_makespan < swarm[best].best_makespan) {
249                     best = index;
250                 }
251             }
252             for (int index = 0; index < size; ++index) {
253                 if (options.topology == "global") {
254                     guide[index] = best;
255                     continue;
256                 }
257                 // Anneau : l'information circule de voisin en voisin, ce qui
258                 // retarde la convergence de tout l'essaim vers un meme point.
259                 int chosen = index;
260                 for (int offset = -options.neighbors;
261                      offset <= options.neighbors; ++offset) {
262                     const int other = ((index + offset) % size + size) % size;
263                     if (swarm[other].best_makespan <
264                         swarm[chosen].best_makespan) {
265                         chosen = other;
266                     }
267                 }
268                 guide[index] = chosen;
269             }
270             return best;
271         };
```

l. 246–251 on repère la meilleure particule de l'essaim (`best`).

l. 253–256 topologie globale : tout le monde a `best` pour guide.

l. 259–267 anneau : on regarde les particules de `index - k` à `index + k` et on garde celle qui a le meilleur souvenir.

l. 262 le modulo referme le cercle : avec 30 particules, les voisines de la particule 0 sont 28, 29, 1 et 2.

7.2 Le redémarrage partiel

Si le meilleur de l'essaim ne s'améliore plus pendant 200 tours (`--restart`), l'essaim a convergé. On garde sa meilleure particule et on relance toutes les autres au hasard, avec de nouvelles vitesses. Elles repartent explorer d'autres zones, et le meilleur planning n'est jamais perdu.

src/pso.cpp, lignes 342 à 348 — mesurer la stagnation

```
342             const int current = swarm[best_index].best_makespan;
343             if (current < swarm_best) {
344                 swarm_best = current;
345                 stagnation = 0;
346             } else {
347                 ++stagnation;
348             }
```

l. 343–345 le meilleur de l'essaim a progressé : le compteur repart à zéro.

l. 347 sinon, un tour de plus sans progrès.

src/pso.cpp, lignes 365 à 375 — le redémarrage partiel

```
365             // CLPSO entretient deja sa diversite : seul l'anneau redemarre.
366             if (!comprehensive && options.restart > 0 &&
367                 stagnation >= options.restart) {
368                 // Redemarrage partiel : l'essaim a converge, on garde sa
369                 // meilleure particule et on relance toutes les autres.
370                 const int keep = compute_guides();
371                 for (int index = 0; index < size; ++index) {
372                     if (index != keep) randomize(swarm[index]);
373                 }
374                 stagnation = 0;
375             }
```

l. 366–367 seulement pour les essaims en anneau, après `restart` = 200 tours sans progrès.

l. 370 `keep` est la meilleure particule de l'essaim.

l. 371–373 toutes les autres sont retirées au hasard avec `randomize`.

Nous avons aussi essayé de relancer les particules *autour* de la meilleure (position du record plus un bruit gaussien). C'était moins bon : l'essaim a besoin de vraie diversité, pas d'une variation sur le même thème.

7.3 CLPSO : l'apprentissage global

Dans le PSO à apprentissage global (*Comprehensive Learning PSO*, Liang *et al.*, 2006), une particule n'a plus de guide unique. Pour **chaque dimension** d, elle choisit un « exemple » : avec une probabilité $P_c(i)$, le meilleur souvenir d'une autre particule tirée par tournoi (la meilleure de deux tirées au hasard), sinon le sien.

$$v_{i,d} \leftarrow w\, v_{i,d} + c\, r\, \left(p_{e_i(d),\,d} - x_{i,d}\right), \qquad P_c(i) = 0{,}05 + 0{,}45\,\frac{e^{10\,i/(N-1)} - 1}{e^{10} - 1}, \qquad c = 1{,}49445$$

Les exemples sont tirés à nouveau quand la particule ne progresse plus pendant 7 tours. Comme chaque particule mélange des morceaux de plusieurs autres, l'essaim ne converge jamais d'un bloc. C'est plus lent, mais cela explore beaucoup mieux. CLPSO entretient lui-même sa diversité : on ne le redémarre pas.

src/pso.cpp, lignes 195 à 224 — choisir les exemples d'une particule

```
195         const auto choose_exemplars = [&](int owner) {
196             Particle& particle = swarm[owner];
197             const double learning = size <= 1 ? 0.0 :
198                 0.05 + 0.45 * (std::exp(10.0 * owner / (size - 1)) - 1.0) /
199                                (std::exp(10.0) - 1.0);
200             std::uniform_int_distribution<int> other(0, size - 1);
201             const auto tournament = [&]() {
202                 int first = other(random);
203                 int second = other(random);
204                 if (size > 1) {
205                     while (first == owner) first = other(random);
206                     while (second == owner) second = other(random);
207                 }
208                 return swarm[first].best_makespan <=
209                        swarm[second].best_makespan ? first : second;
210             };
211             particle.exemplar.assign(dimension, owner);
212             bool learns = false;
213             for (int key = 0; key < dimension; ++key) {
214                 if (unit(random) < learning) {
215                     particle.exemplar[key] = tournament();
216                     learns = true;
217                 }
218             }
219             if (!learns && size > 1) {
220                 std::uniform_int_distribution<int> any_key(0, dimension - 1);
221                 particle.exemplar[any_key(random)] = tournament();
222             }
223             particle.stale = 0;
224         };
```

l. 197–199 la probabilité d'apprentissage $P_c(i)$: 0,05 pour la première particule, 0,5 pour la dernière. Certaines particules restent fidèles à elles-mêmes, d'autres empruntent beaucoup.

l. 201–210 le tournoi : deux autres particules tirées au hasard, on garde celle qui a le meilleur souvenir.

l. 211 par défaut, chaque dimension apprend de la particule elle-même.

l. 213–218 pour chaque dimension, avec la probabilité $P_c(i)$, l'exemple devient le gagnant d'un tournoi.

l. 219–222 si aucune dimension n'a été tirée, on en force une : la particule doit apprendre de quelqu'un.

src/pso.cpp, lignes 295 à 308 — la mise à jour CLPSO

```
295             if (comprehensive) {
296                 if (particle.exemplar.empty() || particle.stale >= 7) {
297                     choose_exemplars(index);
298                 }
299                 for (int key = 0; key < dimension; ++key) {
300                     const double target =
301                         swarm[particle.exemplar[key]].best_position[key];
302                     double speed = inertia * particle.velocity[key] +
303                         1.49445 * unit(random) *
304                             (target - particle.position[key]);
305                     speed = std::clamp(speed, -max_velocity, max_velocity);
306                     particle.velocity[key] = speed;
307                     particle.position[key] += speed;
308                 }
```

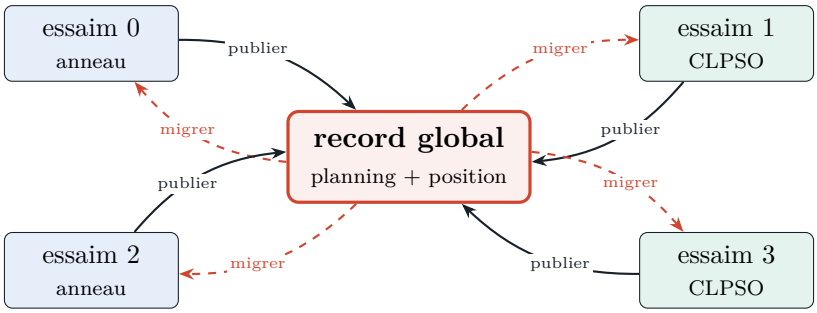
l. 296–298 on retire les exemples quand la particule stagne depuis 7 tours.

l. 300–301 la cible de la dimension `key` est le meilleur souvenir de *son* exemple : des dimensions différentes suivent des particules différentes.

l. 302–304 un seul terme d'attraction au lieu de deux, avec $c = 1{,}49445$.

7.4 Le multi-essaims hétérogène

Le programme lance **un essaim par cœur** du processeur (`--threads 0`). Avec la topologie `mixed`, un essaim sur deux tourne en anneau avec redémarrage, les autres en CLPSO. Les mesures de la section 9 montrent que CLPSO est meilleur jusqu'à 15×15 et l'anneau au-delà. Le mélange est donc robuste quelle que soit la taille des instances du concours.



… jusqu'à un essaim par cœur (12 sur la machine de test)

FIGURE 6 – Les essaims tournent en parallèle et ne partagent que le record. Toutes les 50 itérations (`--migrate`), la pire particule de chaque essaim adopte la position du record global.

`src/pso.cpp`, lignes 277 à 281 — quelle variante pour quel essaim ?

```
277         // "mixed" alterne les deux variantes d'un thread a l'autre : CLPSO
278         // explore mieux les petites instances, l'anneau converge plus vite
279         // sur les grandes, et la migration partage le record entre eux.
280         const bool comprehensive = options.topology == "cl" ||
281             (options.topology == "mixed" && worker_id % 2 == 1);
```

l. 280–281 en mode `mixed`, les essaims de numéro impair (1, 3, 5…) tournent en CLPSO, les essaims pairs en anneau.

`src/pso.cpp`, lignes 350 à 363 — la migration

```
350             if (options.migrate > 0 && iteration % options.migrate == 0 &&
351                 best_makespan.load() < swarm_best) {
352                 // Multi-essaims : la pire particule adopte le record global.
353                 int worst = 0;
354                 for (int index = 1; index < size; ++index) {
355                     if (swarm[index].best_makespan > swarm[worst].best_makespan) {
356                         worst = index;
357                     }
358                 }
359                 std::lock_guard<std::mutex> lock(best_mutex);
360                 swarm[worst].position = global_best_position;
361                 swarm[worst].best_position = global_best_position;
362                 swarm[worst].best_makespan = global_best.makespan;
363             }
```

l. 350–351 tous les 50 tours, et seulement si un autre essaim a fait mieux que celui-ci.

l. 353–358 on cherche la pire particule de l'essaim.

l. 359 on verrouille le record global pendant qu'on le copie (un autre thread pourrait le modifier au même moment).

l. 360–362 la pire particule prend la position et le makespan du record : elle va attirer ses voisines vers cette zone.

`src/pso.cpp`, lignes 378 à 388 — lancer les essaims en parallèle

```
378     // Un essaim par coeur : plus d'essaims, c'est plus de diversite pour le
379     // meme temps de calcul.
380     const int threads = options.threads > 0 ? options.threads
381         : std::max(1, static_cast<int>(std::thread::hardware_concurrency()));
382     std::vector<std::thread> workers;
383     for (int worker_id = 0; worker_id < threads; ++worker_id) {
384         workers.emplace_back(worker, worker_id);
385     }
386     for (auto& worker_thread : workers) {
387         worker_thread.join();
388     }
```

l. 380–381 `--threads 0` : autant d'essaims que de cœurs (`hardware_concurrency`).

l. 383–385 chaque thread exécute la fonction `worker`, c'est-à-dire un essaim complet, avec son numéro.

l. 386–388 on attend que tous les essaims aient fini (fin du budget).

8 Parallélisme et fiabilité

— **Partage du record.** Le makespan du record est un `std::atomic`, lu sans verrou à chaque évaluation. Le planning et la position du record sont protégés par un `std::mutex`, pris seulement quand un essaim bat le record. Les threads ne se ralentissent donc presque jamais.

— **Hasard reproductible par essaim.** Chaque thread a son propre générateur (`mt19937_64`), initialisé à partir de `--seed` et du numéro du thread.

— **Budget de temps.** Le temps est mesuré avec une horloge monotone (`steady_clock`), insensible aux changements d'heure du système : 55 secondes par défaut pour une limite d'une minute.

— **Rien n'est perdu.** Le meilleur planning est écrit dans le JSON au plus toutes les 2 secondes pendant la recherche, puis une dernière fois à la fin. Si le programme est interrompu, le dernier record est sur le disque.

— **Vérification.** `tools/validate.py` relit l'instance et le JSON, et contrôle les trois contraintes et le makespan, indépendamment du code C++.

`src/pso.cpp`, lignes 146 à 163 — publier un nouveau record

```
146     const auto publish = [&](const std::vector<double>& position, int makespan,
147                              int worker_id) {
148         if (makespan >= best_makespan.load()) return;
149         std::lock_guard<std::mutex> lock(best_mutex);
150         if (makespan >= global_best.makespan) return;
151         evaluate(instance, position, options, &global_best);
152         global_best_position = position;
153         best_makespan.store(global_best.makespan);
154         std::cout << "[progression] temps_ms=" << elapsed_ms()
155                   << " makespan=" << global_best.makespan
156                   << " thread=" << worker_id << std::endl;
157         write_if_due(instance, global_best, options, last_write, false);
158         if (log) log << elapsed_ms() << ';' << global_best.makespan << '\n';
159         if (options.target_makespan > 0 &&
160             global_best.makespan <= options.target_makespan) {
161             target_reached.store(true);
162         }
163     };
```

l. 148 test rapide sans verrou : si ce n'est pas mieux que le record, on s'arrête tout de suite. C'est le cas de presque toutes les évaluations.

l. 149–150 sinon, on verrouille et on revérifie : entre-temps, un autre thread a peut-être trouvé mieux.

l. 151–153 on construit le planning complet et on met à jour le record.

l. 154–156 la ligne `[progression]` affichée dans le terminal.

l. 157–158 sauvegarde du JSON et de la courbe de convergence (`--log`).

l. 159–161 avec `--target`, on prévient tous les threads qu'ils peuvent s'arrêter.

src/pso.cpp, lignes 119 à 125 — l'échéance

```
119     const auto start = Clock::now();
120     // --time 0 signifie "pas de limite". Dans ce cas, --target peut arreter
121     // la recherche lorsqu'un makespan donne est atteint.
122     const auto deadline = options.seconds == 0.0
123         ? Clock::time_point::max()
124         : start + std::chrono::duration_cast<Clock::duration>(
125               std::chrono::duration<double>(options.seconds));
```

l. 119 l'heure de départ, lue sur l'horloge monotone `steady_clock`.

l. 122–125 l'échéance = départ + `--time` secondes ; avec `--time 0`, il n'y a pas d'échéance. Chaque essaim teste `Clock::now() < deadline` à chaque tour (ligne 282).

src/pso.cpp, lignes 101 à 115 — sauvegarder sans ralentir

```
101 void write_if_due(const Instance& instance, const Schedule& schedule,
102                   const SolverOptions& options, Clock::time_point& last_write,
103                   bool force) {
104     const auto now = Clock::now();
105     // Une ecriture periodique protege le meilleur resultat si le programme
106     // est interrompu, sans ralentir chaque evaluation.
107     if (!force && now - last_write < std::chrono::seconds(2)) {
108         return;
109     }
110     std::filesystem::create_directories(options.output_directory);
111     write_schedule_json(schedule, instance,
112         (std::filesystem::path(options.output_directory) /
113          (instance.name + ".json")).string());
114     last_write = now;
115 }
```

l. 107–109 si la dernière écriture date de moins de 2 secondes, on n'écrit pas (sauf `force`, utilisé à la toute fin).

l. 110–113 le fichier s'appelle `<nom de l'instance>.json`, dans le dossier `--out`, qui est créé s'il n'existe pas.

src/main.cpp, lignes 82 à 92 — le programme principal

```
82         const Instance instance = read_instance(input);
83         const auto started = std::chrono::steady_clock::now();
84         const Schedule schedule = solve(instance, options);
85         const double elapsed = std::chrono::duration<double>(
86             std::chrono::steady_clock::now() - started).count();
87         std::cout << "makespan=" << schedule.makespan
88                   << " temps_secondes=" << elapsed << '\n';
89     } catch (const std::exception& error) {
90         std::cerr << "Erreur: " << error.what() << '\n';
91         return 1;
92     }
```

l. 82 lecture de l'instance (section 3).

l. 84 tout le travail : les essaims tournent jusqu'à l'échéance.

l. 87–88 la ligne finale `makespan=... temps_secondes=...`.

l. 89–91 toute erreur (fichier introuvable, instance mal formée, option inconnue) est affichée proprement et le programme renvoie le code 1.

Le format de sortie. Pour l'atelier 3×3 avec l'ordre A :


```
{
  "makespan": 11,
  "jobs_schedule": {
    "Job_0": [{"machine": 0, "start": 0, "end": 3},
              {"machine": 1, "start": 4, "end": 6},
              {"machine": 2, "start": 7, "end": 11}],
    "Job_1": [{"machine": 1, "start": 0, "end": 4},
              {"machine": 2, "start": 4, "end": 7},
              {"machine": 0, "start": 8, "end": 10}],
    "Job_2": [{"machine": 2, "start": 0, "end": 2},
              {"machine": 0, "start": 3, "end": 8},
              {"machine": 1, "start": 8, "end": 11}]
  }
}
```

src/schedule.cpp, lignes 143 à 160 — écrire ce JSON

```
143     output << "{\n  \"makespan\": " << schedule.makespan
144            << ",\n  \"jobs_schedule\": {\n";
145     for (int job = 0; job < instance.jobs; ++job) {
146         output << "    \"Job_" << job << "\": [";
147         const auto& operations = schedule.jobs.at(static_cast<std::size_t>(job));
148         for (std::size_t operation = 0; operation < operations.size();
149              ++operation) {
150             const auto& value = operations[operation];
151             if (operation != 0) {
152                 output << ", ";
153             }
154             output << "{\"machine\": " << value.machine
155                    << ", \"start\": " << value.start
156                    << ", \"end\": " << value.end << "}";
157         }
158         output << "]" << (job + 1 == instance.jobs ? "\n" : ",\n");
159     }
160     output << "  }\n}\n";
```

l. 143–144 l'en-tête avec le makespan.

l. 145–146 une entrée `"Job_0"`, `"Job_1"`… par job.

l. 151–156 chaque étape devient `{"machine", "start", "end"}`, séparées par des virgules.

l. 158 pas de virgule après le dernier job, sinon le JSON serait invalide.

9 Résultats

Les mesures portent sur 7 instances de la bibliothèque JSPLIB, de 10×10 à 30×20, dont l'optimum (ou la meilleure solution connue) est publié. Le critère est l'**écart** au meilleur connu :

$$\text{écart} = \frac{\text{makespan obtenu} - \text{meilleur connu}}{\text{meilleur connu}}.$$

9.1 Convergence sur l'instance des slides

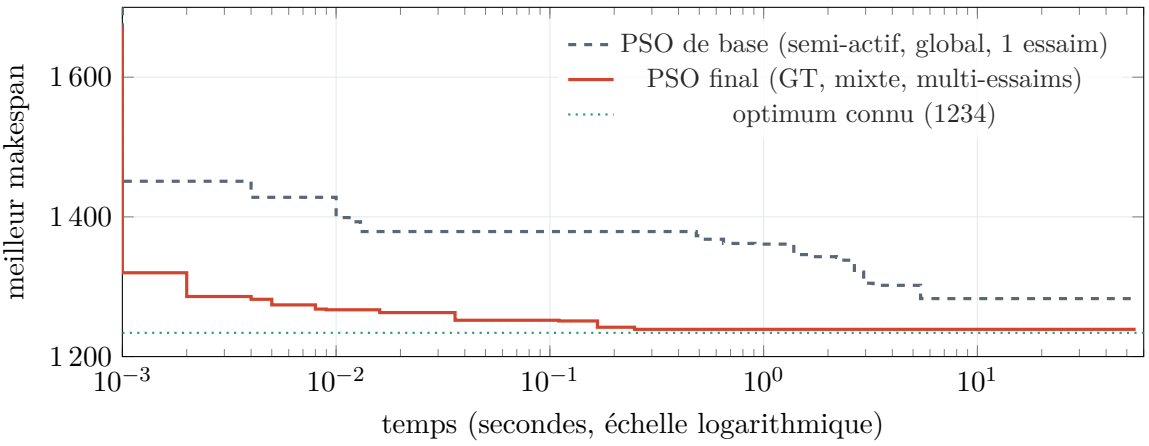


FIGURE 7 – Évolution du meilleur makespan sur abz5 (l'instance `10_10` des slides), en 55 secondes. Chaque marche est un nouveau record. Le PSO de base trouve son dernier record (1283, soit 4,0 % au-dessus de l'optimum) après 5,4 s puis n'avance plus. Le PSO final atteint 1239 (0,4 %) dès 0,25 s : sur une instance de cette taille, ses 12 essaims trouvent très vite un planning proche de l'optimum, puis plafonnent eux aussi.

9.2 Ce qu'apporte chaque mécanisme

Chaque ligne ajoute un mécanisme à la précédente. Les mesures sont faites en 10 secondes sur 2 à 3 graines aléatoires, ce qui permet de comparer rapidement. La dernière ligne est mesurée sur le vrai budget de 55 secondes.

Table 5 – Écart moyen au meilleur connu sur les 7 instances.

Configuration	Écart moyen
Décodeur semi-actif, topologie globale (version de départ)	22,3 %
Ancien décodeur « GT » approximatif	10,4 %
Nouveau décodeur Giffler-Thompson paramétré	6,8 %
+ topologie en anneau	6,4 %
+ redémarrage après 200 tours sans progrès	5,9 %
+ voisinage de 2 et $\delta = 0{,}2$	5,1 %
+ 12 essaims indépendants (un par cœur)	4,7 %
+ migration du record toutes les 50 itérations	4,2 %
+ essaims mixtes anneau / CLPSO	3,8 %
Configuration finale, 55 secondes	**3,3 %**

9.3 Résultats par instance

Table 6 – Version de départ (10 s, 1 essaim) contre version finale (55 s, 12 essaims).

Instance	Taille	Meilleur connu	Départ	Final	Makespan final
abz5 (`10_10`)	10×10	1234	3,8 %	0,4 %	1239
ft10	10×10	930	7,5 %	2,0 %	949
la21	15×10	1046	9,4 %	0,9 %	1055
la26	20×10	1218	8,4 %	**0,0 %**	**1218**
ta01	15×15	1231	10,4 %	4,1 %	1282
ta21	20×20	1644	14,7 %	5,2 %	1729
ta41	30×20	2018	18,5 %	10,7 %	2234
Moyenne			**10,4 %**	**3,3 %**	

Table 7 – Anneau contre CLPSO seuls (55 s, 1 essaim, moyenne de 3 graines) : des profils complémentaires, d'où le choix d'essaims mixtes.

	abz5	ft10	la21	la26	ta01	ta21	ta41
Anneau + redémarrage	0,41	2,15	2,96	0,93	4,93	**6,97**	**12,09**
CLPSO	0,49	2,15	**1,15**	**0,00**	**4,01**	7,89	14,87

9.4 Ce que nous avons appris

— **Le décodeur compte autant que l'essaim.** Changer de décodeur a fait plus que tous les réglages d'essaim réunis : c'est lui qui définit l'espace où les particules cherchent.

— **La stagnation se combat par la diversité, pas par le temps.** Doubler le temps de calcul n'a presque rien changé (5,17 % → 5,09 %), alors que les topologies, les redémarrages et les essaims multiples ont fait gagner plusieurs points.

— **Il faut mesurer avant de garder une idée.** La relance autour du record semblait prometteuse ; elle faisait moins bien qu'une relance aléatoire. Les petits réglages (nombre de particules, inertie, `vmax`) donnaient des écarts plus petits que le bruit de mesure (±0,3 %).

— **Limite.** Les très grandes instances restent difficiles (ta41 à 10,7 %) : avec 600 clés par particule, un PSO pur peine à explorer un espace aussi vaste en une minute.

10 Utilisation

Compiler (une fois, en mode optimisé) :

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure
```

Lancer une instance avec les réglages par défaut, qui sont les meilleurs trouvés :

```
./build/jsp_pso instances/abz5 --out resultats
python3 tools/validate.py instances/abz5 resultats/abz5.json
```

Table 8 – Les options de la ligne de commande.

Option	Défaut	Rôle
`--time S`	55	budget en secondes (0 = sans limite)
`--threads N`	0	nombre d'essaims, 0 = un par cœur
`--topology`	mixed	`ring`, `global`, `cl` (CLPSO) ou `mixed`
`--neighbors K`	2	voisins de chaque côté dans l'anneau
`--restart N`	200	tours sans progrès avant relance (0 = jamais)
`--migrate N`	50	période de migration du record (0 = jamais)
`--particles N`	30	particules par essaim
`--vmax V`	0,2	vitesse maximale par dimension
`--decoder`	gt	`gt` (Giffler-Thompson) ou `semi` (semi-actif)
`--delta D`	0,2	fenêtre du décodeur `gt`
`--seed N`	0	graine du hasard
`--out DIR`	.	dossier du JSON de sortie
`--target M`		s'arrêter dès qu'un makespan $\leq M$ est trouvé
`--log FICHIER`		écrire la courbe de convergence (CSV)

Tests automatiques. `ctest` vérifie la lecture de l'exemple du cours et de huit fichiers volontairement défectueux, le décodage de l'ordre A (makespan 11), l'écriture du JSON, et que le PSO retrouve l'optimum 11 sur l'atelier 3×3. Une compilation avec *sanitizers* (`-DJSP_SANITIZE=ON`) détecte en plus les erreurs mémoire et les comportements indéfinis.